of code that works with multiple providers. Should you need to switch providers for cost, performance, or availability reasons, the change is a simple configuration update rather than a major architectural refactor.

Streaming responses provides a consistent and unified streaming API that works across different AI providers. This feature directly addresses the challenge of building real-time, dynamic user experiences. With streaming, you can display content as it becomes available from the LLM, dramatically improving perceived performance and user engagement. The SDK handles the complexities of managing the connection and processing the incremental data chunks, freeing you to focus on the UI.

While the SDK offers additional features like state management, React server component support, and the ability to generate structured data, these will be explored in subsequent chapters as we add more complex functionality. For now, understanding these two core features is all you need to get started.

3.1.2 A strategic approach to integration

Introducing a new framework like the Vercel AI SDK into an existing application requires a thoughtful strategy. While our current application is small and the changes will be minimal, it's a perfect opportunity to apply core software development principles that ensure maintainability and scalability for the long run. Let's start by discussing the key guidelines we will follow during our integration process.

SEPARATION OF CONCERNS

Separation of concerns is a crucial principle in software development that promotes modularity, maintainability, and reusability. It involves breaking down a system into distinct components, each with a well-defined responsibility. By ensuring that your application's components are modularized and have clear boundaries, you can easily swap out or modify specific components without affecting the entire codebase.

One effective way to achieve separation of concerns is by distinguishing between intents and actions. An *intent* represents a high-level feature or functionality that your application provides, such as autocomplete, generation, or suggestion. On the other hand, an *action* is the specific implementation or the concrete steps taken to fulfill that intent. Actions often involve calling APIs, interacting with databases, or invoking specific models or algorithms.

For example, let's consider a feature that generates text based on user input. The intent of this feature is to provide text generation capability. The action associated with this intent would be making a request to a specific API or model that can generate the desired text. In this case, we can define an intent called `generateText` and map it to the corresponding action of making a chat completion request to generate the text.

Figure 3.1 illustrates the basic concepts of separating intents from actions. In the diagram, the left part represents the intent `generateText`, while the right part represents the action `chatCompletion`. The arrow between them signifies the connection point that bridges the intent with its corresponding action.